

Module 4 Normalisation:

You build the college database as **one** big table. Everything in it — student, course, instructor, grade. One table, no joins, nice and simple.

Three weeks later Prof. Rao gets married and changes his surname.

You now have to edit 400 rows. You miss two.

Why + What

That's not bad luck. It's the table design biting you.

The problem is that one table is holding facts about **three different subjects at once** — students, courses, and enrollments. And when unrelated facts share a row, editing one thing means touching many rows.

Before we fix it, we need a way to say precisely which fact decides which. That's what a functional dependency is.

A functional dependency means: if you know the value of column A, then the value of column B is fixed. We write it $A \rightarrow B$, and read it "A decides B".

Normalization is the process of splitting tables so that every fact is stored exactly once.

How It Works (with the easiest example)

Look at one wide table holding roll_no, name, course, title, instructor, grade.

1. **Find who decides what.** Give me a roll_no and I can tell you the name — so roll_no $\rightarrow$ name. Give me a course and I can tell you the title and the instructor — so course $\rightarrow$ title, instructor.
2. **Find what the grade needs.** Does roll_no alone decide the grade? No — Aisha has different grades in different courses. Does course alone? No. You need **both**: (roll_no, course) $\rightarrow$ grade.
3. **Now spot the mismatch.** The key of this table is (roll_no, course). But title only needed course. So title is sitting in a table whose key is bigger than what it actually depends on. That mismatch is what causes the repetition.
4. **Watch it repeat.** Aisha and Ben are both in DBMS 101, so "Databases" and "Rao" get stored twice. Add 400 students and it's stored 400 times.
5. **Name the three ways it hurts you:**
 - **Update anomaly** — Rao changes his name, and you must edit every row he appears in. Miss one and the database now claims two different names.
 - **Insert anomaly** — a new course opens with no students yet. There's no row to put it in, so the course simply can't exist.
 - **Delete anomaly** — Cara drops OS101, and hers was the only OS101 row. Delete it and the whole course disappears with her.

Pause here. Which anomaly hits you if the college wants to record a new instructor who hasn't been assigned a course yet?

Insert anomaly. There's no row to hold him, because a row can only exist if a student-course pair exists.

Visualize

4.1 WHY ONE BIG TABLE HURTS

ENROLLMENT_BIG · EVERYTHING IN ONE TABLE

ROLL_NO	NAME	COURSE	TITLE	INSTRUCTOR	GRADE
101	Aisha	DBMS101	Databases	Rao	A
102	Ben	DBMS101	Databases	Rao	B
103	Cara	OS101	Operating Sys	Iyer	A

"Rao" and "Databases" stored twice already

UPDATE ANOMALY

Rao changes his name.

You must edit every row he appears in.

✗ Miss one and he has two names.

INSERT ANOMALY

A new course opens.

No student has enrolled yet.

✗ No row to hold it. Course can't exist.

DELETE ANOMALY

Cara drops OS101.

Her row is the only OS101 row.

✗ Delete it and OS101 vanishes too.

Notice: all three problems come from one cause — mixed-up facts in one table.

Notice: all three problems come from one cause — mixed-up facts in one table.

Common Confusion

People read $A \rightarrow B$ as "A causes B" or "A comes before B." It means neither. It only means: **for one value of A, there is exactly one value of B.**

Test it with real rows. If you ever find two rows with the same A but different B, then $A \rightarrow B$ is false. That's the whole check.

Second thing: a functional dependency is a rule about the **real world**, not about the rows you happen to have today. $\text{name} \rightarrow \text{roll_no}$ might look true if all your current students have different names. It's still wrong, because two students *could* share a name. Ask "could this ever collide," not "has it collided yet."

Interview Angle

1. "What is a functional dependency?" — If you know A, B is fixed: $A \rightarrow B$. Give a quick example like $\text{roll_no} \rightarrow \text{name}$ and say why the reverse doesn't hold.
2. "Name the three anomalies and give an example of each." — Update (edit many rows for one fact), insert (can't add a course with no students), delete (removing a student deletes the course). Interviewers want the examples, not the labels.
3. "Why do these anomalies happen in the first place?" — Because one table is storing facts about more than one subject, so a single fact ends up repeated across rows.

Quiz: skipped — reinforced above.

Recap: A functional dependency $A \rightarrow B$ means A decides B, and when a table mixes facts from different subjects it repeats data — causing update, insert and delete anomalies.

4.2 Normal Forms: 1NF, 2NF, 3NF — Core • 18 min

Hook

You know the big table is broken. Fine. Split it.

But split it *how*? Into two tables? Five? Which columns go where?

You need a checklist, not a hunch.

Why + What

Normal forms are that checklist. Each one is a rule your table either passes or fails, and each rule kills one specific kind of repetition.

They stack. You reach 2NF only after 1NF, and 3NF only after 2NF. So this is one staircase, not three separate ideas.

1NF: every cell holds a single value. 2NF: no column depends on only part of the key. 3NF: no non-key column depends on another non-key column.

For interviews, **3NF is the finish line**. Almost every real database stops there.

How It Works (with the easiest example)

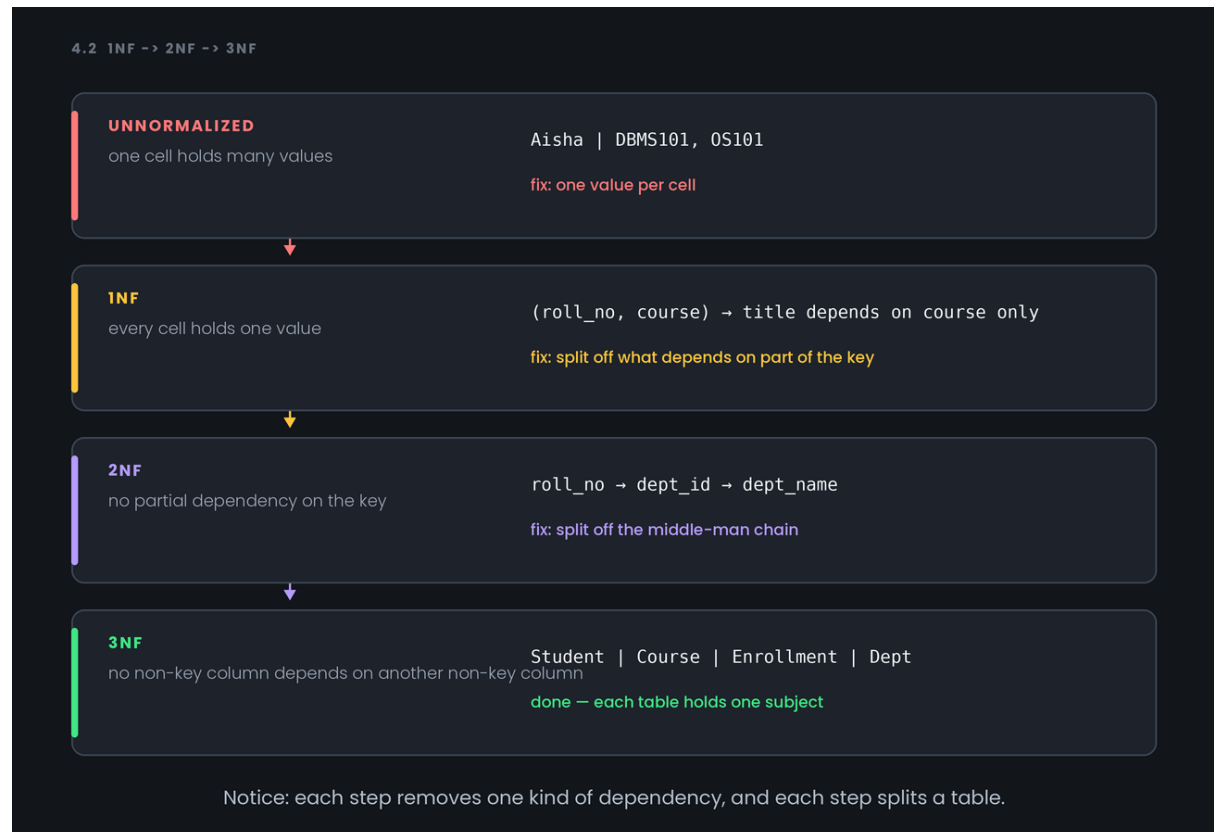
Start from the wide table you met in 4.1, key (roll_no, course).

1. **Fail 1NF first.** Someone stored Aisha's courses as one cell: DBMS101, OS101. You can't search or join on that. **Fix:** one value per cell. Aisha now gets two rows, one per course. **Now in 1NF.**
2. **Check 2NF — the partial dependency.** The key is (roll_no, course). But name depends on roll_no alone, and title depends on course alone. Each depends on only **part** of the key. That's a **partial dependency**. **Fix:** move them out. Student(roll_no, name), Course(course, title), and Enrollment(roll_no, course, grade) keeps what genuinely needs both. **Now in 2NF.**
3. **Check 3NF — the middle-man.** Suppose Student also holds dept_id and dept_name. Here roll_no $\rightarrow$ dept_id, and dept_id $\rightarrow$ dept_name. So roll_no decides dept_name only by going *through* dept_id. That indirect hop is a **transitive dependency**. **Fix:** pull the chain apart. Student(roll_no, name, dept_id) and Dept(dept_id, dept_name). **Now in 3NF.**
4. **Read the result.** Four tables, and each one describes exactly one subject: a student, a course, an enrollment, a department.
5. **Test the fix.** Prof. Rao changes his name now — you edit **one** row in Course. A new course with no students? Insert it into Course; nobody needs to enroll first. Cara drops OS101? Delete one enrollment row; the course stays. **All three anomalies gone.**

Pause here. Before reading on, say which normal form each of these breaks: (a) a phone_numbers cell holding "999, 888", (b) Enrollment(roll_no, course, grade, title).

(a) 1NF — two values in one cell. (b) 2NF — title depends on course alone, which is only part of the key.

Visualize



Notice: each step removes one kind of dependency, and each step splits a table.

Common Confusion

2NF and 3NF sound the same. They're not. Look at *what* the column depends on:

- **2NF problem** — a column depends on **part of the key**. (title needs only course, not the full (roll_no, course).)
- **3NF problem** — a column depends on **another normal column**. (dept_name needs dept_id, which isn't a key at all.)

Part of the key → 2NF. A non-key column → 3NF.

Useful shortcut: **if your table's key is a single column, it's automatically in 2NF**, because a single column has no "part" to partly depend on. So 2NF problems only ever appear in tables with composite keys — like Enrollment.

The old exam line is worth remembering too: *every non-key column must depend on the key, the whole key, and nothing but the key*. "The key" is 1NF, "the whole key" is 2NF, "nothing but the key" is 3NF.

Interview Angle

1. *"Explain 1NF, 2NF and 3NF with an example."* — Walk one table through all three, like the steps above. A single example carried through beats three definitions.
2. *"What's a partial dependency? What's a transitive dependency?"* — Partial: depends on part of a composite key. Transitive: depends on a non-key column, so you reach it in two hops. One example each.
3. *"A table has a single-column primary key. Can it violate 2NF?"* — No. There's no partial key to depend on, so it's automatically in 2NF. This one catches a lot of candidates.
4. *"How far do you normalize in practice?"* — 3NF for almost everything. Going further is rare, and going backwards is a deliberate performance decision, which is 4.3.

Quiz

Q1. Enrollment(roll_no, course, grade, course_title) with key (roll_no, course) violates: (a) 1NF (b) 2NF (c) 3NF (d) nothing **Answer: (b)** — course_title depends on course alone, which is only part of the key. That's a partial dependency.

Q2. Student(roll_no, name, dept_id, dept_name) with key roll_no violates: (a) 1NF (b) 2NF (c) 3NF (d) nothing **Answer: (c)** — dept_name depends on dept_id, a non-key column. Transitive dependency. It cannot be a 2NF problem, because the key is one column.

Q3 (scenario). A table stores (roll_no, hobbies) with Aisha's row reading 101 | chess, painting. Which form does it break, what's the fix, and how many of Aisha's rows exist afterwards?

Answer: It breaks **1NF** — one cell, two values. The fix is one value per cell, so Aisha gets **two rows**: 101 | chess and 101 | painting. Note that roll_no alone is no longer unique here — the key becomes (roll_no, hobby).

Recap: 1NF gives one value per cell, 2NF removes columns that depend on part of the key, 3NF removes columns that depend on other non-key columns — and 3NF is where most real designs stop.

4.3 BCNF & Denormalization — Core • 14 min

Hook

Your table passes 3NF. You're pleased.

Then someone updates a row, and a fact you never touched changes meaning.

3NF wasn't quite enough.

Why + What

3NF has a small gap. It allows a column that isn't a key to decide another column, as long as that other column happens to be part of a key. Rare, but real — and when it bites, the repetition comes back.

BCNF closes that gap with a single, stricter sentence.

BCNF (Boyce–Codd Normal Form): every time some column decides another, the deciding column must be a key.

And there's the opposite direction to know as well. **Denormalization means deliberately putting repeated data back in, to make reads faster.**

How It Works (with the easiest example)

Take a table (student, subject, teacher), where a teacher teaches exactly one subject, and a student can study a subject with any teacher.

1. **Check the key.** (student, subject) identifies a row. Both are key columns, so there are no non-key columns at all — which means it passes 3NF automatically.
2. **Find the odd dependency.** teacher → subject. Rao teaches DBMS, so knowing the teacher tells you the subject.
3. **Ask the BCNF question.** Is teacher a key of this table? No. So a non-key column is deciding something. **It fails BCNF**, even though it passed 3NF.
4. **Fix it by splitting.** Teaches(teacher, subject) and StudentTeacher(student, teacher). Now every decider is a key of its own table.
5. **Now go the other way on purpose.** Your dashboard shows "student name + course title + instructor" and joins four tables every time. It's slow. So you copy course_title back into a reporting table, accepting the repetition to remove the joins. That's **denormalization** — a knowing trade, not a mistake.

Visualize

4.3 BCNF & DENORMALIZATION

3NF BUT NOT BCNF

STUDENT	SUBJECT	TEACHER
Aisha	DBMS	Rao
Ben	DBMS	Rao

teacher → subject
a non-key column decides another column,
and teacher is not a key here.

BCNF: EVERY DECIDER MUST BE A KEY

THE TRADE-OFF

MORE NORMALIZED

- no repeated facts
- no anomalies
- more tables → more joins

DENORMALIZED ON PURPOSE

- fewer joins → faster reads
- used for reports/dashboards
- repeated facts come back

Notice: normalize by default — denormalize only when a slow query proves you must.

Notice: normalize by default — denormalize only when a slow query proves you must.

Common Confusion

"BCNF is just a stricter 3NF" is true but too vague to use. Here's the sharp version:

- **3NF** allows a non-key column to decide something, **if** the thing it decides is part of a key.
- **BCNF** allows no exceptions at all. Every decider must be a key.

So every BCNF table is in 3NF, but not the reverse — the same nesting shape you saw with keys in 3.2.

Second confusion: people think denormalization means "bad design" or "lazy." It doesn't. The difference is **intent and evidence**:

- Never normalized in the first place → a mistake.
- Normalized properly, then measured a slow query, then deliberately traded → engineering.

Say it that way in an interview and you sound like someone who's shipped something.

Last one: **normalizing is not free**. More tables means more joins, and joins cost time. That cost is exactly why 3NF is the usual stopping point rather than pushing to the highest form available.

Interview Angle

1. *"What's the difference between 3NF and BCNF?"* — BCNF requires every decider to be a key, with no exception for key columns. Add that BCNF is stricter, so every BCNF table is in 3NF.
2. *"When would you denormalize?"* — When a measured read is too slow and the join count is the cause — typically reporting or dashboard tables. Stress that you measure first and accept the repetition knowingly.
3. *"What's the cost of over-normalizing?"* — More tables, more joins, slower reads, and more complex queries to write and maintain.
4. *"Is a table in 3NF always in BCNF?"* — No. Give the (student, subject, teacher) case: it passes 3NF but fails BCNF because teacher decides subject without being a key.

Quiz

Q1. Every table in BCNF is also in: (a) 1NF only (b) 2NF only (c) 1NF, 2NF and 3NF (d) none of these **Answer: (c)** — the forms stack, and BCNF sits above 3NF.

Q2. You copy course_title into a reporting table to avoid a join. This is: (a) a normalization error (b) deliberate denormalization (c) a 2NF violation you must fix (d) a foreign key **Answer: (b)** — a knowing trade of extra repetition for faster reads.

Recap: BCNF closes 3NF's last gap by demanding that every deciding column be a key, while denormalization deliberately reverses normalization to buy read speed — measured first, never by accident.

Module 5: SQL

Module 5: SQL

Same story throughout: student **Aisha** (roll 101) takes **DBMS 101**, taught by **Prof. Rao**, and gets grade **A**. Tables in play: Student, Course, Enrollment, Instructor.

5.1 SQL Basics: SELECT, WHERE, ORDER BY — Core • 16 min

Hook

Four modules in, you've designed a good database. Tables split properly, keys in place, constraints holding.

Now someone actually asks it a question: "Who got an A?"

You need a language.

Why + What

SQL is that language. And unlike most languages you've written, you don't tell it **how** to find the answer — you describe **what** you want, and the database figures out the how.

That's why the same query works whether the table has 10 rows or 10 million.

SQL is the standard language for talking to a relational database. It has a few families: DDL builds the tables, DML changes the rows, and DQL — really just SELECT — asks questions.

Learn the families once so you never blank when an interviewer asks "is TRUNCATE DDL or DML?"

How It Works (with the easiest example)

Let's ask the database who got an A.

1. **Pick the table — FROM.** FROM Enrollment. Everything starts here. No table, no query.
2. **Throw out rows you don't want — WHERE.** WHERE grade = 'A'. This works on **one row at a time**: keep it or drop it.
3. **Choose the columns — SELECT.** SELECT name, grade. WHERE picks rows, SELECT picks columns. Different jobs.
4. **Sort the survivors — ORDER BY.** ORDER BY name. Add DESC to reverse it. Sorting happens last, on whatever is left.
5. **Read the whole thing:**

sql

SELECT name, grade

FROM Enrollment

WHERE grade = 'A'

ORDER BY name;

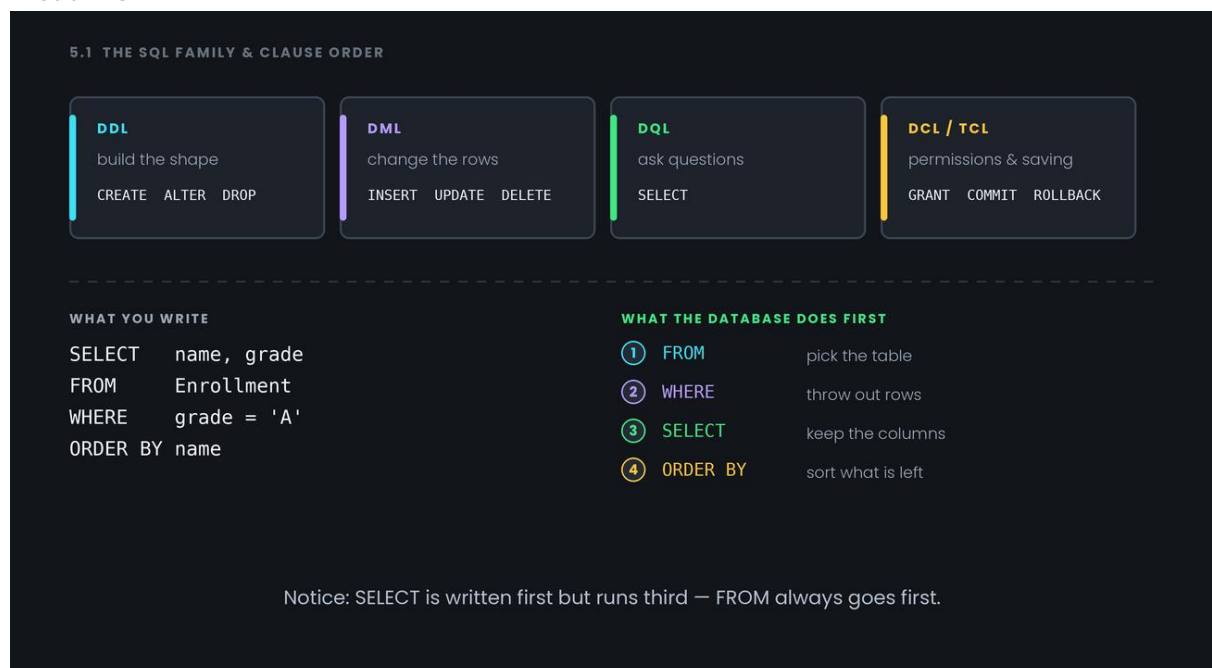
Now the twist. You **wrote** SELECT first, but the database **runs** FROM first, then WHERE, then SELECT, then ORDER BY.

That's not trivia. It explains a real error you'll hit: you can't use a column nickname from SELECT inside WHERE, because when WHERE runs, SELECT hasn't happened yet.

Pause here. SELECT name AS student_name FROM Student WHERE student_name = 'Aisha'. Will this run?

No. WHERE runs before SELECT, so the nickname student_name doesn't exist yet. Use WHERE name = 'Aisha'.

Visualize



Notice: SELECT is written first but runs third — FROM always goes first.

Common Confusion

DELETE vs TRUNCATE vs DROP. All three remove things, but they're not interchangeable:

- **DELETE** — removes rows, and you can add a WHERE to pick which. It's DML, so it can be rolled back.
- **TRUNCATE** — removes **all** rows fast, no WHERE allowed. It's DDL.
- **DROP** — removes the whole table, structure and all. Also DDL.

DELETE empties the box row by row. TRUNCATE tips the box out. DROP throws the box away.

Second one: **NULL doesn't behave like a value.** WHERE grade = NULL returns nothing at all — not even rows where the grade really is NULL. NULL means "unknown", and comparing anything to unknown gives unknown, not true. You must write WHERE grade IS NULL.

Interview Angle

1. *"What's the difference between DELETE, TRUNCATE and DROP?"* — DELETE removes chosen rows and can be rolled back; TRUNCATE removes all rows quickly; DROP removes the table itself. Mention that DELETE is DML while the other two are DDL.
2. *"In what order does SQL actually run a query?"* — FROM, WHERE, GROUP BY, HAVING, SELECT, ORDER BY. Then give the payoff: that's why a SELECT alias can't be used in WHERE.
3. *"How do you find rows where a column has no value?"* — IS NULL, never = NULL, because a comparison with unknown never returns true.

Quiz: skipped — reinforced above.

Recap: SQL describes what you want, not how to get it — FROM picks the table, WHERE filters rows, SELECT picks columns, ORDER BY sorts, and the run order is not the writing order.

5.2 Joins — Core • 20 min

Hook

You normalized in Module 4 and it worked beautifully. Student names live in Student. Course codes live in Enrollment. Nothing repeats.

Now your boss wants one report: **student name next to course code**.

That data is in two different tables. Now what?

Why + What

This is the bill for normalization, and it comes due here. You split tables to stop repetition; joins are how you put the pieces back together for one query.

A join combines rows from two tables by matching a value in one against a value in the other — almost always a foreign key matching a primary key.

How It Works (with the easiest example)

Keep it tiny so you can see every row. Two students, two enrollments — deliberately mismatched.

Student: 101 Aisha, 104 Dev Enrollment: 101 DBMS101, 999 OS101

So Dev enrolled in nothing, and roll 999 doesn't exist as a student.

1. **INNER JOIN — only matches survive.** Match Student.roll_no with Enrollment.roll_no. Only 101 exists on both sides. **One row out.** Dev disappears. Roll 999 disappears.

sql

```
SELECT s.name, e.course
```

```
FROM Student s
```

```
INNER JOIN Enrollment e ON s.roll_no = e.roll_no;
```

2. **LEFT JOIN — keep everything on the left.** Every student stays, matched or not. Aisha gets her course. Dev has no match, so his course comes back **NULL**. **Two rows out.**
3. **RIGHT JOIN — keep everything on the right.** Every enrollment stays. The 999 row survives with name = NULL, because no student matched it.
4. **FULL JOIN — keep everything on both sides.** Aisha's matched row, Dev with a NULL course, and 999 with a NULL name. **Three rows out.**
5. **Read the NULLs as a message.** A NULL in join output never means "empty data." It means *"nothing on the other side matched."* That's why LEFT JOIN plus WHERE course IS NULL is the standard way to find students who enrolled in nothing.

Pause here. Which join would you use to list students who have **no** enrollments at all?

LEFT JOIN from Student, then WHERE e.course IS NULL. Keep every student, then keep only the ones whose match came back empty.

Visualize

5.2 JOINS

STUDENT	
ROLL_NO	NAME
101	Aisha
104	Dev

ENROLLMENT	
ROLL_NO	COURSE
101	DBMS101
999	OS101

Dev has no enrollment. Roll 999 has no student.

INNER JOIN		
only rows that match on both sides		
ROLL_NO	NAME	COURSE
101	Aisha	DBMS101

LEFT JOIN		
all students, matched or not		
ROLL_NO	NAME	COURSE
101	Aisha	DBMS101
104	Dev	NULL

RIGHT JOIN		
all enrollments, matched or not		
ROLL_NO	NAME	COURSE
101	Aisha	DBMS101
NULL	NULL	OS101

FULL JOIN		
everything from both sides		
ROLL_NO	NAME	COURSE
101	Aisha	DBMS101
104	Dev	NULL
NULL	NULL	OS101

Notice: NULL marks the side that had nothing to match with.

Notice: NULL marks the side that had nothing to match with.

Common Confusion

LEFT vs RIGHT confuses people far more than it should. "Left" just means the table written **before** the JOIN keyword. That's it. And every RIGHT JOIN can be rewritten as a LEFT JOIN by swapping the table order — which is why you'll rarely see RIGHT JOIN in real code.

Second: **forgetting the ON condition**. Write a join with no ON and you get a **cross join** — every row paired with every other row. Two tables of 1,000 rows give you a million rows. It's not an error message; it's just a very wrong, very slow answer.

Third, and this one is subtle: **filtering in WHERE vs in ON**. With a LEFT JOIN, putting a condition on the right table in WHERE quietly turns it back into an INNER JOIN — because the unmatched rows have NULLs, and NULL fails the WHERE test, so they get dropped. If you want to keep unmatched rows while filtering, the condition belongs in ON.

Interview Angle

1. *"Explain INNER, LEFT, RIGHT and FULL JOIN."* — Walk two small tables through all four and say how many rows each returns. Concrete row counts beat definitions here.
2. *"How do you find rows in table A with no match in table B?"* — LEFT JOIN from A, then WHERE b.some_column IS NULL. This is asked constantly.
3. *"What's a self join and when would you use it?"* — Joining a table to itself, using two aliases. Classic case: an Employee table where manager_id points at another row in the same table.
4. *"What happens if you forget the ON clause?"* — A cross join: every row against every row. Fine on purpose, disastrous by accident.

Quiz

Q1. Student has 5 rows, Enrollment has 3 rows, and only 2 enrollments match a real student. An INNER JOIN returns: (a) 5 (b) 3 (c) 2 (d) 15 **Answer: (c)** — only matched pairs survive an inner join.

Q2. In a LEFT JOIN, a NULL in a right-hand column means: (a) the data was deleted (b) no matching row existed on the right (c) an error (d) the value is zero **Answer: (b)** — the left row was kept, but nothing on the right matched it.

Q3 (scenario). You write a LEFT JOIN from Student to Enrollment, then add WHERE e.grade = 'A'. Students with no enrollment vanish from the result. Why, and how do you keep them? **Answer:** Unmatched students get e.grade = NULL, and NULL = 'A' is never true, so WHERE drops them — the LEFT JOIN has effectively become an INNER JOIN. To keep them, move the condition into the join: LEFT JOIN Enrollment e ON s.roll_no = e.roll_no AND e.grade = 'A'.

Recap: A join matches rows across tables — INNER keeps only matches, LEFT and RIGHT keep one whole side, FULL keeps both, and any NULL in the output means that side had nothing to match.

5.3 Aggregate Functions, GROUP BY & HAVING — Core • 16 min

Hook

Nobody asks "show me all 4,000 enrollments."

They ask: *how many students per course? what's the average grade? which courses are over-subscribed?*

Those are questions about **groups of rows**, not rows.

Why + What

Everything so far worked one row at a time. WHERE looks at a row and keeps or drops it. But "how many students in DBMS 101" can't be answered by looking at one row — you have to look at a pile of them together.

An aggregate function squeezes many rows into one value: COUNT, SUM, AVG, MIN, MAX. GROUP BY splits rows into buckets so the aggregate runs once per bucket. HAVING then filters those buckets.

How It Works (with the easiest example)

Three enrollment rows: DBMS101 A, DBMS101 B, OS101 A. Question: how many students per course?

1. **Aggregate with no grouping.** SELECT COUNT(*) FROM Enrollment gives **3**. One number for the whole table. Useful, but not per-course.
2. **Split into buckets — GROUP BY.** GROUP BY course makes two buckets: a DBMS101 bucket holding two rows, and an OS101 bucket holding one.
3. **Run the aggregate per bucket.** COUNT(*) now runs once inside each bucket: DBMS101 → 2, OS101 → 1. **Each bucket collapses into a single output row.**

sql

```
SELECT course, COUNT(*) AS students
```

```
FROM Enrollment
```

```
GROUP BY course;
```

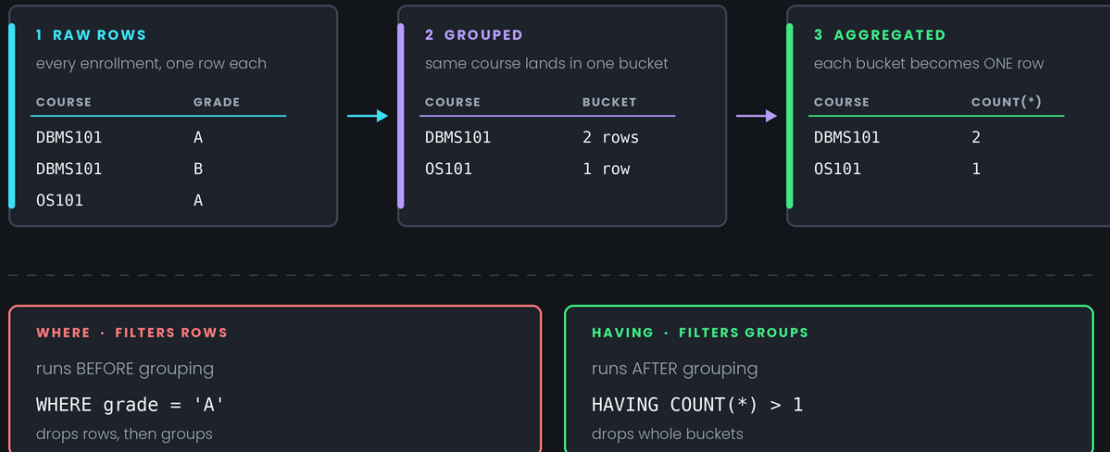
4. **Filter the buckets — HAVING.** Want only courses with more than one student? HAVING COUNT(*) > 1 keeps DBMS101 and drops OS101. Note you couldn't have used WHERE here — the count didn't exist yet when WHERE ran.
5. **Use both together.** WHERE grade = 'A' first drops rows, *then* grouping happens on what's left. So WHERE shrinks the input; HAVING shrinks the output.

Pause here. A course has 3 enrollments but only 1 with grade A. With WHERE grade='A' ... GROUP BY course, what count comes back for that course?

1. WHERE ran first and removed the other two rows, so only one row ever reached the bucket.

Visualize

5.3 GROUP BY PIPELINE



Notice: WHERE works on rows, HAVING works on the buckets they became.

Notice: WHERE works on rows, HAVING works on the buckets they became.

Common Confusion

WHERE vs HAVING is the most-asked question in this whole topic. One line settles it: **WHERE runs before grouping, HAVING runs after.** So WHERE can't see aggregate results, and HAVING can.

The other classic error: **selecting a column that isn't grouped.** This fails:

sql

```
SELECT course, name, COUNT(*) FROM Enrollment GROUP BY course;
```

The DBMS101 bucket holds two different students, so which name should it print? There's no sensible answer, so the query is rejected. **Rule: every column in SELECT must either be in GROUP BY, or be wrapped in an aggregate.**

Last one: **COUNT(*) vs COUNT(column)**. COUNT(*) counts rows. COUNT(grade) counts rows where grade **is not NULL**. On a table with ungraded enrollments, those two give different numbers — and interviewers know it.

Interview Angle

1. "Difference between WHERE and HAVING?" — WHERE filters rows before grouping, HAVING filters groups after. Add that HAVING can use COUNT/SUM and WHERE cannot.
2. "Why can't you put a non-grouped column in SELECT?" — Because a bucket holds many rows and the database can't pick which value to show. Every SELECT column must be grouped or aggregated.

3. *"COUNT() versus COUNT(column)?"** — COUNT(*) counts all rows; COUNT(column) skips NULLs in that column.
4. *"Can you use both WHERE and HAVING in one query?"* — Yes, and it's common: WHERE trims the input rows, HAVING trims the resulting groups.

Quiz

Q1. To keep only groups with more than 5 rows, you use: (a) WHERE COUNT() > 5 (b) HAVING COUNT() > 5 (c) SELECT COUNT() > 5 (d) ORDER BY COUNT() **Answer: (b)** — WHERE runs before the count exists.

Q2. A grade column has 10 rows, 3 of them NULL. COUNT(*) and COUNT(grade) return: (a) 10 and 10 (b) 10 and 7 (c) 7 and 7 (d) 7 and 10 **Answer: (b)** — COUNT(*) counts rows; COUNT(grade) ignores NULLs.

Recap: Aggregates squeeze many rows into one value, GROUP BY makes the buckets they run inside, and HAVING filters those buckets — while WHERE filters rows before any of it happens.

5.4 Subqueries & Set Operations — Core • 16 min

Hook

"Show me students who scored above the class average."

You can't write that in one pass. You don't know the average until you've looked at every row — but you need the average to decide which rows to keep.

You need an answer before you can ask your question.

Why + What

That's exactly what a subquery does. It runs first, produces an answer, and hands it up to the query that needed it.

A subquery is a query written inside another query. The inner one runs first, and its result is used by the outer one.

And a related tool: **set operations stack two separate result lists into one — UNION, INTERSECT and EXCEPT.**

How It Works (with the easiest example)

Question: show the names of students taking DBMS 101. Names live in Student, enrollments live in Enrollment.

1. **Write the inner question first.** Who's in DBMS 101? SELECT roll_no FROM Enrollment WHERE course = 'DBMS101' → gives 101, 102.
2. **Feed it upward with IN.**

sql

```
SELECT name FROM Student
```

```
WHERE roll_no IN (SELECT roll_no FROM Enrollment WHERE course = 'DBMS101');
```

The database runs the inside, gets (101, 102), and the outer query becomes a plain WHERE roll_no IN (101, 102).

3. **Know the three shapes a subquery returns.**

- **One value** — use with =: WHERE grade > (SELECT AVG(grade) FROM ...)
- **A list of values** — use with IN, as above
- **A whole table** — use in the FROM clause, as a temporary table

4. **Now set operations.** These stack two finished result lists:

- **UNION** — everything from both, duplicates removed (UNION ALL keeps duplicates and is faster)
- **INTERSECT** — only what appears in both
- **EXCEPT** — in the first list but not the second

5. **The one rule for all three.** Both lists must have the **same number of columns, in the same order, with matching types**. Names don't have to match; shapes do.

Pause here. You want students enrolled in DBMS 101 but **not** OS 101. Which tool — a join, a subquery, or a set operation?

Any of the three works, but EXCEPT reads most directly: the DBMS101 list, EXCEPT the OS101 list.

Visualize

5.4 SUBQUERIES & SET OPERATIONS

A SUBQUERY RUNS FIRST, THEN HANDS ITS ANSWER UP

STEP 1 · INNER QUERY

```
SELECT roll_no FROM Enrollment
WHERE course = 'DBMS101'
```

↓ gives 101, 102

STEP 2 · OUTER QUERY

```
SELECT name FROM Student
WHERE roll_no IN ( 101, 102 )
```

the inner query must return one column
when the outer query uses IN.

SET OPERATIONS · STACK TWO RESULT LISTS

UNION
everything, duplicates removed
 $A + B$

INTERSECT
only what is in both lists
 $A \text{ and } B$

EXCEPT
in the first list, not the second
 $A \text{ minus } B$

all three need the same column count and types on both sides.

Notice: a subquery answers a question first; a set operation stacks two answers.

Notice: a subquery answers a question first; a set operation stacks two answers.

Common Confusion

Subquery vs join — people think one must be "correct." Often both work, and modern databases optimise them into the same plan. Pick by readability: joins when you need **columns from both tables** in the output, subqueries when you only need a **filter condition**. In the example above, we wanted names only, never enrollment columns — so the subquery reads better.

Second: **NOT IN and NULL are a trap**. If the subquery returns even one NULL, NOT IN returns **no rows at all** — because "is this value not equal to unknown" can never be answered true. NOT EXISTS doesn't have this problem, which is why experienced people prefer it.

Third: **UNION vs UNION ALL**. UNION removes duplicates, which means sorting the whole result — real work. If you know there are no duplicates, or you don't care, UNION ALL is meaningfully faster. Saying this unprompted looks good.

Interview Angle

1. *"What's a correlated subquery?"* — One that references a column from the outer query, so it re-runs once per outer row. Regular subqueries run just once; correlated ones are slower.
2. *"When would you use a subquery instead of a join?"* — When you only need a filter and no columns from the second table. Say that both are often equally fast, and the choice is about readability.
3. *"Difference between UNION and UNION ALL?"* — UNION removes duplicates and pays a sorting cost; UNION ALL keeps everything and is faster.
4. *"Why can NOT IN return nothing unexpectedly?"* — If the inner list contains a NULL, every comparison becomes unknown and no row qualifies. Use NOT EXISTS instead.

Quiz

Q1. A subquery that references a column from the outer query is called: (a) nested (b) correlated (c) recursive (d) inline **Answer: (b)** — correlated, and it re-runs for each outer row.

Q2. UNION differs from UNION ALL in that it: (a) is faster (b) allows different column counts (c) removes duplicate rows (d) only works on two tables **Answer: (c)** — and removing duplicates is exactly why it's slower.

Recap: A subquery runs first and hands its answer to the outer query, while UNION, INTERSECT and EXCEPT stack two result lists that must share the same column shape.

5.5 Views — Core • 10 min

Hook

The exam office needs to see grades. They must **not** see fee balances — that's in the same Student table.

You could write them a careful query and hope they never edit it. Or you could hand them a window that only shows what they're allowed to see.

Why + What

You already met the idea in 1.3 without the name: the **external level**, where each user sees only their own slice of the database. A view is how you actually build that slice.

A view is a saved query with a name. It stores no rows of its own — every time you read it, its query runs again on the real tables underneath.

How It Works (with the easiest example)

1. **Write the query you want people to have.** Join Student and Enrollment, select name, course, grade — and simply don't include fees_due.
2. **Save it with a name.**

sql

```
CREATE VIEW student_grades AS
```

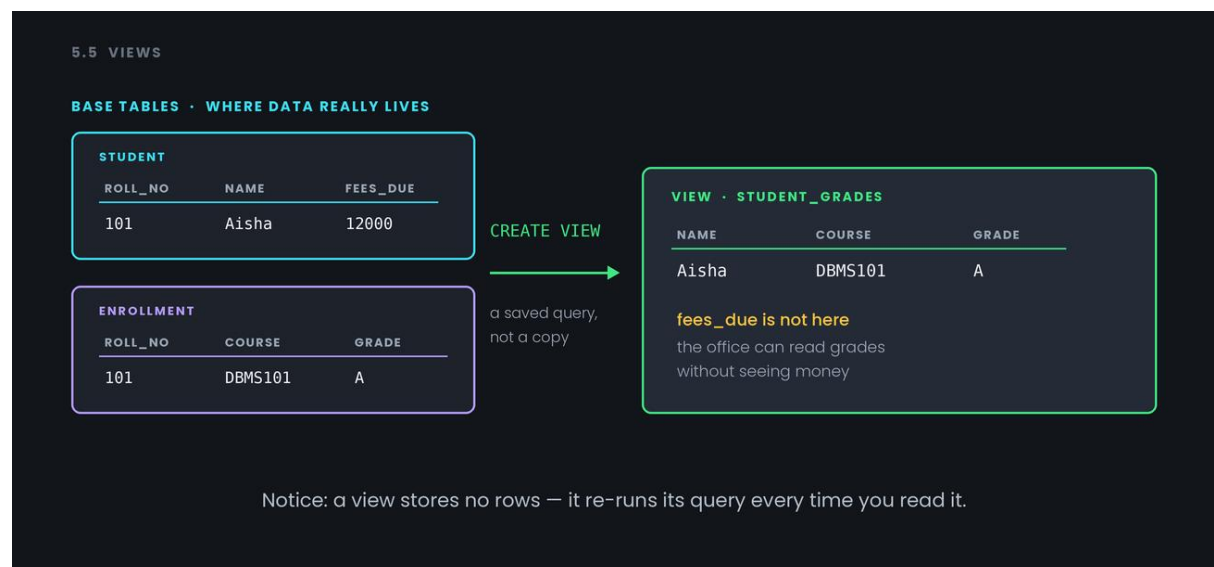
```
SELECT s.name, e.course, e.grade
```

```
FROM Student s
```

```
JOIN Enrollment e ON s.roll_no = e.roll_no;
```

3. **Use it like a table.** `SELECT * FROM student_grades WHERE grade = 'A'` works exactly as if it were one. That's the whole appeal — the join is hidden inside.
4. **Notice what's not there.** fees_due is unreachable through this view. Give the office access to the view instead of the table, and the money column is invisible to them.
5. **Watch it stay current.** Update a grade in Enrollment and the view shows the new value immediately — because it holds no copy. It re-runs its query each time you read it.

Visualize



Notice: a view stores no rows — it re-runs its query every time you read it.

Common Confusion

The big one: **people think a view stores data.** It doesn't. It's a saved query, so it costs storage of essentially nothing and costs query time every read.

The exception worth naming: a **materialized view** *does* store the result physically. It's fast to read but goes stale until it's refreshed. Normal view = always fresh, re-computed. Materialized view = fast, possibly stale.

Second: **can you update through a view?** Sometimes. A simple view over one table usually accepts updates. A view with joins, GROUP BY, or DISTINCT usually doesn't — because the database can't work out which underlying row you meant to change.

Interview Angle

1. *"What is a view and why use one?"* — A saved query used like a table. Reasons: hide sensitive columns, hide complex joins, and give different users different slices.
2. *"Does a view store data?"* — No, it re-runs its query each time. A materialized view does store results, trading freshness for speed.
3. *"Can you INSERT or UPDATE through a view?"* — Simple single-table views, usually yes. Views with joins or aggregates, usually no, because the target row is ambiguous.

Quiz: skipped — reinforced above.

Recap: A view is a named saved query that stores no rows — it hides columns and complexity, always reflects the current data, and only a materialized view actually stores results.